Booking an Appointment

This guide covers booking an appointment in Boulevard. You'll learn how to query for the service menu, add a BookableItem to the cart, search for available dates and times, submit credit card information, and complete the checkout.

Step 1. Choose a location

The likely first step in any booking process is to have the client select which location of your business the client is interested in booking with.

> **Note:** This step is only necessary if your business has multiple locations.

```
query {
  business {
    locations(first: 20) {
      edges {
        node {
          id
          name
          tz

          address {
            line1
            line2
            city
            state
            zip
          }
        }
      }
    }
  }
}
```

Step 2. Create a cart and display the menu

After a desired location has been identified, a cart should be created. The cart provides an interface for building a composable booking experience.

> **Note:** The ID of the cart can be stored in the client's cookies or local storage to provide a resumable booking experience that persists between page loads.

```
mutation {
  createCart(input: {
    locationId: "1"
  }) {
    cart {
      id

      availableCategories {
        name

        availableItems {
          id
          name
        }
      }
    }
  }
}
```

▶ SHOW RESULTS

Step 3. Add an item to the cart

When a menu item is selected, you may then request the specific staff members (if applicable) and the cart will use that staff's price and durations. If no staff member is specified, first available will be used.

```
query {
```

```
    availableItem(id: "1") {
      id

      ... on CartAvailableBookableItem {
        name
        description
        staffVariants {
          id
          price
          duration

          staff {
            firstName
            lastName
            bio
          }
        }
      }
    }
  }
}
```

Then, to add the item to the cart, collect the `itemId` (and the optional `itemStaffVariantId`) from the prior step, and mutate the cart with the desired service:

```
mutation {
  addCartSelectedBookableItem(input: {
    id: "1",
    itemId: "2",
    itemStaffVariantId: "3"
  }) {
    cart {
      id
    }
  }
}
```

Step 4. Find a time

> See the **Available dates and times** guide for more information, including solutions for common use cases.

Once a cart has at least one Bookable Item added, you may request the bookable dates according to the query below:

```
query {
  cartBookableDates(
    id: "1",
    searchRangeLower: "2020-09-01",
    searchRangeUpper: "2020-09-02",
    tz: "America/Los_Angeles"
  ) {
    date
  }
}
```

For each date that you would like to list the available times, you may execute the following query:

```
query {
  cartBookableTimes(
    id: "1",
    searchDate: "2020-09-01",
    tz: "America/Los_Angeles"
  ) {
    id
    score
    startTime
  }
}
```

Once the desired date and time is chosen, call the reserveCartBookableItems mutation, as such:

> **Note:** This call may fail if the time is no longer available.

```
mutation {
  reserveCartBookableItems(input: {
    id: "1",
    bookableTimeId: "2"
  }) {
    cart {
      id
    }
  }
}
```

Step 5. Collect client information

The collection of client information can happen at any point during the booking process, including during the creation of the cart itself.

```
mutation {
  updateCart(input: {
    id: "1",
    clientInformation: {
      email: "john.doe@gmail.com",
      firstName: "John",
      lastName: "Doe",
      phoneNumber: "+13105555555"
    }
  }) {
    cart {
      id
    }
  }
}
```

Step 6. Collect credit card information

To secure the booking, a valid credit card should be submitted prior to the checkoutCart mutation.

> **Note:** The Tokenization API must be used to securely redact credit card information prior to the use of this API call.

```
mutation {
  addCartCardPaymentMethod(input: {
    id: "1",
    token: "bbe6d8d9-1f45-4989-a1ba-90a014051340",
    select: true
  }) {
    cart {
      id
    }
  }
}
```

Step 7. Complete the checkout

As the final action in the booking process, the checkoutCart mutation may be used. This will check all necessary prerequisites, such as the presence of client contact information, and will return descriptive errors for each failing requirement. If successful, the completedAt property will be set indicated the timestamp at which the booking successfully completed.

> **Note:** The client will automatically receive a confirmation email, and (optionally) the staff member may be notified according to the communication preferences set within Boulevard. Once this mutation has been successfully made, no further changes to the cart is allowed.

```
mutation {
  checkoutCart(input: {
    id: "1"
  }) {
    cart {
      id
      completedAt
    }
  }
}
```

Group Booking

The API supports the concept of booking multiple appointments for a group. The flow is very similar to what is outlined above, except that you must add guests and corresponding bookable items to the Cart before checking it out. Checkout will result in an appointment for every guest, linked to the parent appointment for the owner of the Cart. Start time is shared by all guests, meaning they must all come in at the same time.

Step 1. Create a cart and display the menu

```
mutation {
  createCart(input: {
    locationId: "1"
  }) {
    cart {
      id

      availableCategories {
        name

        availableItems {
          id
          name
        }
      }
    }
  }
}
```

Step 2. Add Bookable Item to the Cart for the Owner

```
mutation {
  addCartSelectedBookableItem(input: {
    id: "1",
    itemId: "2"
  }) {
    cart {
      id
    }
  }
}
```

Step 3. Add Guest(s) to the cart

```
mutation {
  createCartGuest(
    input: {
      id: "1",
      email: "guest@example.com",
      firstName: "John",
      lastName: "Smith",
      phoneNumber: "+14802895555"
    }
  ) {
    cart {
      id
      guests {
        id
        firstName
        lastName
        email
      }
    }
  }
}
```

This will return the cart with the guest and their Id. You can get the guests at anytime off the cart by querying the `guests` field. You'll need the id returned on the guest to supply in the `addCartSelectedBookableItem` mutation below.

Step 4. Now Add a bookable item to the Cart for the Guest

Note: The itemGuestId field is supplied this time when adding the bookableItem. This explicitly tells the Cart that this bookable item is for the previously added guest.

```
mutation {
```

```
    id: "1",
    itemId: "3",
    itemGuestId: "4"

  }) {
    cart {
      id
    }
  }
}
```

The remaining flow is the same as outlined in the standard booking flow. **Steps 3 and 4 can be repeated for every guest before proceeding to checkout the Cart.**

Returning Client Booking

Clients may decide to return to your custom booking experience to book another appointment. If your integration accepts client information that can be used to authenticate and find a client in Boulevard, then you have the opportunity to retrieve additional context for a client, such as previously used payment methods and redeemable vouchers.

There are two ways to authenticate the Cart with a specific client. Please refer to each option below.

1. Using an authenticated client token directly

If your integration is able to reliably obtain the client in Boulevard using information supplied by your users, such as a login page, then you can generate an authenticated token directly and use it to gain context about that client from the Cart.

Most commonly, you will need to find the returning client by email, name, or phone number. This must be done using the Admin API and requires your API secret. See the **ClientConnection** in the Admin API.

Once you've obtained the client's ID, you can generate an **authenticated client token**. See the **Authenticated Client API Access** section of the token generation guide.

Please note: This option is not well suited to integrations that are not able to manage their own client authentication.

a. Create the cart using the authenticated client token

By simply creating the cart with the authenticated client token, additional context will become available for the client immediately. This option is great if you have a custom login page prior to the start of the booking flow.

> **Note:** availablePaymentMethods will return previously used payment methods by the authenticated client. These can be used with the selectCartPaymentMethod mutation to apply the previously used card to the cart.

```
mutation {
  createCart(input: {
    locationId: "1"
  }) {
    cart {
      id

      availablePaymentMethods {
        id
        name

        ... on CartItemCardPaymentMethod {
          cardBrand
          cardExpMonth
          cardExpYear
          cardLast4
        }

        ... on CartItemVoucherPaymentMethod {
          expiresOn
          availableCount
        }
      }
    }
  }
}
```

See below results. Both credit card and available vouchers are able to be retrieved as payment methods and displayed in your booking flow

> **Note:** voucher payment methods are not yet available in the API to be used at booking, only at checkout within the Boulevard Dashboard

b. Take ownership of the cart after it's been created

The **takeCartOwnership** mutation is a more explicit way of authenticating the client with the cart. Just like the prior option, the authenticated client token must be used as an Authorization header in order for the mutation to succeed. This option is great if you'd like to present a custom login page later on in the booking flow, perhaps after a service has been selected.

```
mutation {
  takeCartOwnership(input: {
    id: "1"
  }) {
    cart {
      id

      availablePaymentMethods {
        id
        name
      }
    }
  }
}
```

2. Take ownership of the cart via email or SMS

Rather than having to manage authenticating clients in your integration, another option is to authorize the client's ownership of the cart via email or SMS. This can be more convenient as it allows the client to gain ownership of the cart without needing to know their password.

This involves 2 steps:

1. Send a cart ownership code to the user via email or SMS

A cart ownership code first needs to be sent to the client using either email or SMS. You may use the corresponding mutations **sendCartOwnershipCodeByEmail** or **sendCartOwnershipCodeBySms** to send the cart ownership code. These mutations also return a `cartOwnershipCodeId` which must be retained until step 2.

Via email

```
  mutation {
    sendCartOwnershipCodeByEmail(input: {
      email: "test@example.com"
    }) {
      cartOwnershipCodeId
    }
  }
```

▶ **SHOW RESULTS**

Via SMS

```
mutation {
  sendCartOwnershipCodeBySms(input: { mobilePhone: "+14842918831" }) {
    cartOwnershipCodeId
  }
}
```

▶ **SHOW RESULTS**

2. Take ownership of the cart using the ownership code

Once the client has received the `cartOwnershipCodeValue` it may now be used to take ownership of the cart. In addition to the `cartOwnershipCodeValue` you'll also need the `cartOwnershipCodeId` (from step 1) and the `cartId`.

```
mutation {
  takeCartOwnershipByCode(input: {
    cartId: "41af7957-2878-475d-91cb-0d01f7b9a5c8",
    cartOwnershipCodeId: "712bd037-c534-415b-bddd-3aea35e33031",
    cartOwnershipCodeValue: "$code_client_received_via_email_or_sms"
  }) {
    cart {
      id
    }
  }
}
```

Service Addons

You can use service addons to upsell or cross sell services. Addons can be configured in Boulevard Dashboard.

Listing & adding addons to cart

To fetch the addons, you should use the `addons` field on the **CartItem** type. This will return a list of **CartAvailableItem** structs.

The relation between the addon and it's base item is encoded in the ID, and it's transparent to the client code. That means that to add an addon to cart, you use typical `addCartSelected{TYPE}Item` mutations (e.g. **cartAddSelectedBookableItem**) and Boulevard backend takes care of creating the correct relationship.

> Reminder: All the IDs coming from the GraphQL APIs should be treated as opaque and their exact structure can always change. Changing the ID structure is not considered a breaking change, so if you want to make sure your code won't break in the future, you should never parse the IDs or rely on their structure.

Finding addons already added to the cart

Once an addon is added to a cart, you can see the `baseBookableItem` field on the **CartBookableItem** associated with that addon. This is a link to the base item.

Special addon behaviour

Addons are subject to some extra rules you should be aware of:

- When a base service is removed from the cart, all the addons for that service are also removed.
- Addons that are already added to a given `CartItem` are not displayed on the `addons` list.
- Addon's staffVariants might be incompatible with the base service. That means that the base service's staff selection might be cleared after adding an addon to the cart.